DS 6021: Intro to Predictive Modeling
Lab 6: Missing Data, Regularization, Logistic Regression

October 23, 2026
Due: October 27, 2026 11:59pm

Materials Required

- Auto.csv – Gas mileage (mpg), horsepower, and other information for cars from model years 1970–1982, from An Introduction to Statistical Learning (ISL). This is a version of the UCI Auto MPG dataset.
- framingham.csv – Framingham heart study, predicting coronary heart disease
- Regularization materials from Week of 10/12 with Jeffrey Blume
- NEW notebook that has been checked into our class github repository: missing_data_and_imputation.ipynb

Activity Overview

For the last few weeks, we have had lectures on missing data and imputation, regularization, and an introduction to logistic regression, but we haven't had the opportunity to practice doing any of these concepts. In this lab, we will practice all of them! In particular, practicing regularization and logistic regression will help you study for the exam.

In Part 1, we review a notebook (missing_data_and_imputation.ipynb) that explores how to handle missing data and different imputation techniques. This is the notebook that we didn't have time to go over in class, and so part of this activity will involve reading the notebook and answering some of the questions posed in the notebook itself.

In Part 2, we will once again use the auto dataset for feature selection and cross validation, but this time we will use regularization. The work you did for Lab 5 will be helpful here.

In Part 3, we will practice doing logistic regression. (This will come up again in subsequent Lab Activities!)

Part 1: Missing Data and Imputation (8 pts)

In class on 10/9, we spent a bit of time discussing missing data, but ran out of time to do an example. We will do that in this part of the lab! This will use the Framingham heart study data, available in our class github repository. Before answering these questions, please check out the missing_data_and_imputation.ipynb notebook from our class github repository and execute each of the cells.

1. **[8 pts]** Notebook review:
    a. [1 pt] One of the cells in the notebook errors out. Why?
    b. [1 pt] What does the correlation plot tell us?
    c. [2 pts] As part of this notebook, we delete 80% of the values of sysBP, so we can simulate the impact of missing data and imputation. Based on how we did the deletion, does this make the missing sysBP data MAR, MNAR, or MCAR? Explain your choice.
    d. [2 pts] In Approach #2, we fit a second model with significantly fewer data points from our baseline (831 vs 4143) and plot the coefficient values for each model. What does this plot tell you about the impact of removing sysBP on coefficient estimates?

e. [2 pts] In Approach #3, we used mean imputation to impute missing values for sysBP. Why might I have chosen to do this using the mean value calculated from the dataset with 80% of the values missing vs the entire dataset we started with at the beginning?

Part 2: Regularization (15 pts)

In this section, we will use the auto dataset from Lab Activity 5 and build upon what you learned last week from Jeffrey. In Lab Activity 5, you did Forward and Backward selection on the five numeric features of the auto data set before predicting mpg. You probably recall that this dataset has a number of different colinear variables, and previously, we looked at keeping or dropping them based on Forward and Backward Feature Selection. This time, we will use regularization to explore how Ridge, Lasso, and ElasticNet effect the model selection process and beta coefficients for each feature.

For this part of the lab, use ["displacement", "horsepower", "weight", "acceleration", "year"] to predict mpg.

Preamble and Setup

There are a few differences between how R does regularization (as we saw in class) and how Python does it. Key points are summarized in the table below:

glmnet(R)	Scikit-learn (Python)	**Meaning**
lambda	alpha	Strength of regularization penalty
alpha	ll_ratio	Mix between ridge (0) and Lasso (1)
cv.glmnet(…)	RidgeCV, LassoCV, ElasticNetCV	Use KFold CV to choose the penalty

Please use the following setup in your code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import ElasticNetCV, LassoCV, LinearRegression, RidgeCV, lasso_path
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

auto = pd.read_csv("../data/Auto.csv", na_values="?").dropna().reset_index(drop=True)

num = ["displacement", "horsepower", "weight", "acceleration", "year"]
X = auto[num]
y = auto["mpg"]

# Same 5-fold CV object for all methods, so the results are comparable
SEED = 200
cv = KFold(n_splits=5, shuffle=True, random_state=SEED)

# Grid of penalty strengths (sklearn's alpha = glmnet's lambda) to search over
# Remember: We don't know what the right penalty is, we are trying to find it!
alphas = np.logspace(-3, 3, 100)
auto.shape
```

1. **[3pts]** Fit the following two models using OLS: 1) Predict mpg using all of the numeric features. 2) Predict mpg using horsepower, weight, and year only. You do not need to use a train/test split or cross validation for this part of the exercise. You DO need to use StandardScaler on the variables.
    a. Output the coefficients for both models in a table. We will be adding to this table in subsequent parts of this question.
    b. Why is standardization required for ridge and lasso regression, when for OLS it changed only how we interpret the coefficients, but not the fit of the model?
2. **[4pts]** Ridge regression
    a. Use RidgeCV(alphas=alphas, cv=cv) in a pipeline with StandardScaler() to fit a ridge model.
    b. Report the penalty (.alpha_) chosen by the ridge regression model and add the coefficients to the table you created in Q1.
    c. Are any of the ridge coefficients exactly zero? Compare the ridge coefficients to the full OLS model from Q1. Which coefficients changed the most?
3. **[4pts]** Lasso regression
    a. Use LassoCV(alphas=alphas, cv=cv) in a pipeline with StandardScaler() to fit a lasso model.
    b. Report the penalty (.alpha_) chosen by the lasso regression model and add the coefficients to the table you created in Q1. Which features did the lasso model drop? (e.g. the coefficient went to zero.)
    c. In general, for a Lasso model that chooses three coefficients, why might we trust this more than those chosen by a Forward/Backward selection with k=3? What limits might the Lasso regression have?
4. [**4pts**] ElasticNet
    a. Use `ElasticNetCV(alphas=alphas, l1_ratio=[0.1, 0.5, 0.9, 1], cv=cv)` in a pipeline with `StandardScaler()`. This lets cross-validation choose both the penalty strength and the ridge/lasso mix.
    b. Report the chosen `alpha_`, the chosen `l1_ratio_`, and the coefficients. Add the coefficients to the table you created in Q1. What does the chosen ` l1_ratio_` tell you?
    c. Looking at all of the approaches in this Part of the lab, which features do the different methods agree are important?

Part 3: Logistic Regression (15 pts)

In this part of the Lab, we will practice doing logistic regression and interpreting model fit.

Guidance for this section: We will be using the Framingham heart study data as we did in class. We will be exploring two different models. The target variable (y) is TenYearCHD

- Model A: Use male + age as your X variables
- Model B: Use male + cigsPerDay as your X variables

Preamble and Setup

In class, we introduced the log loss function, which is used to optimize the beta coefficients for the logistic regression using maximum likelihood. Once these coefficients have been determined, we can write the log loss for the resulting model by summing over the predicted probabilities for each observation. It looks like this:

$$\text{log loss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)]$$

The 1/n factor gives the average log_loss per observation. (In this case, it is the average log loss per patient) sklearn does this averaging by default. This equation will be helpful in doing this part of the homework. We will also spend more time discussing this in class on Monday 10/26.

Please use the following setup in your code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, log_loss
from sklearn.model_selection import StratifiedKFold, cross_val_predict, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Same data preparation as in class: drop rows with any missing values
fram = pd.read_csv("../data/framingham.csv").dropna()
y = fram["TenYearCHD"]

# Same CV object as in class. Stratified folds keep the ~15% CHD rate in every fold.
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Preprocess by scaling numeric variables using Standard Scaler
# You don't need to scale or one hot encode the binary male categorical variable
# For Model A, replace numeric with age, for Model B, replace numeric with cigsPerDay
preprocess = ColumnTransformer(transformers=[
        ("binary", "passthrough", ["male"]),
        ("numeric", StandardScaler(), [numeric]),
    ])
```

1. **[3 pts]** Baselines
    a. What fraction of the patients developed CHD within 10 years?
    b. Suppose a “model” predicts no CHD for every patient. What is its accuracy?
    c. For the “model” in part b, we compute the same probability for each patient, and it is equal to the fraction of patients from part a. What is the log_loss for this “model”?
2. [**4pts**] Fit both models and compute log loss.
    a. Fit Model A and Model B as described above. You do not need to use a holdout set for this part, we will use cross-validation shortly.
    b. For each model, get the predicted probabilities of CHD using predict_proba (as we did in class) and compute the log_loss for each using the sklearn log_loss function.
    c. For Model A, compute the log loss by hand with NumPy. It should match the value from sklearn.
3. **[8 pts]** Now let’s determine which model is better. All of the numbers above are in sample, so we need to use cross-validation for this part.
    a. Use `cross_validate` with the `cv` object from the setup and `scoring=["accuracy", "neg_log_loss", "roc_auc"]` to evaluate both models. Report the mean and standard deviation across the 5 folds for each metric. (Remember that sklearn negates log loss, so flip the sign back.)

b. Compare the CV accuracy of the two models with the baseline accuracy from Q1(b). Is accuracy a useful metric for choosing between these models? Why or why not?
c. Use `cross_val_predict` with `method="predict_proba"` to get out-of-fold predicted probabilities for each model. Plot the confusion matrices for both models at thresholds of 0.5 and 0.2. What do you observe?
d. Based on the log_loss, roc_auc, and confusion matrices, which model would you choose and why? Relate your answer to the log loss rate for the "model" in Q1

Deliverables

For each part, please include the following:

- All code used to complete the activities. This can be turned in as a single Jupyter notebook, multiple Jupyter notebooks, or one or more .py files. Up to you!
- For open ended questions, please include a few sentences or a paragraph answering the question. This text can be included in your Jupyter notebook as a markdown cell or a separate document.
- Per our policy on Generative AI detailed in the syllabus, if you use it to generate code, you need to flag how and where you used it, and how you ascertained the correctness of what you did. You are not allowed to use Generative AI for any of the written explanations or reflection questions. If you did not use Generative AI for any part of the assignment, please state that explicitly.